FraudShiftBench: Deployment-Contract Evaluation for Temporal Graph Fraud Detection

Anonymous Author

***Abstract*—Temporal graph-fraud results are usually reported for a fixed split, graph, and metric, although deployment also fixes when labels arrive, which edges are visible, how the graph is constructed, how many alerts can be reviewed, and what computation is feasible. A model ranking without those conditions has no stable operational meaning. We introduce FraudShiftBench, which represents these conditions as a deployment contract and binds each scoped conclusion to a typed evidence unit. Its support relation checks cell completeness, prediction provenance, paired uncertainty, and resource status before a claim can enter the benchmark. We instantiate the framework with ten-seed evidence on two real financial graphs and four synthetic IBM AML-Data regimes. On Elliptic, changing only strict versus isolated graph access changes the AUPRC leader from GraphSAGE to GCN; their paired mean changes are -0.132 and $+0.094$, respectively, while the feature-only MLP is invariant. In the IBM baseline grid, histogram gradient boosting has the highest mean AUPRC in all eight variant–protocol cells. Across the broader feasible IBM study, however, AUPRC and fixed-threshold F1 select different configurations in 11 of 16 contexts. Within the graph-configuration grid, edge-conditioned GINE leads AUPRC in all four measured Small contexts, but its Medium cells remain outside the ranking after T4 memory exhaustion. Fourteen claim-mutation and evidence-ablation tests produce the specified status transitions, including rejection of incomplete, prediction-missing, and construct-invalid claims. The resulting benchmark is a conditional map of protocol, architecture, construction, metric, scale, and feasibility rather than a global leaderboard. The released artifact contains prediction manifests, evidence locks, executable validators, deterministic analyses, and an explicit record of exclusions and resource boundaries.**

***Index Terms*—Graph fraud detection, anti-money laundering, temporal shift, evaluation protocols, benchmark validity, reproducibility.**

I. Introduction

FRAUD screening is a temporal decision problem. Transactions arrive in order, labels often arrive later, and illicit activity is rare. Relational context can be useful, but its availability depends on how a production graph is maintained at scoring time. Investigators then inspect only a small part of a ranked queue. These conditions make the test set, graph view, and decision rule part of the scientific question, not details to be fixed after a model has been chosen. They also make precision–recall analysis and capacity-aware measures more informative than an isolated AUROC number [1]–[3].

Public financial-graph datasets have enabled increasingly capable detectors. Elliptic introduced temporally indexed Bitcoin transactions [4]; DGraphFin supplied a much larger account graph [5]; and IBM AML-Data made controlled transaction regimes available at several scales [6]. Yet a result on any of these datasets remains conditional. A transductive graph exposes a different information set from a graph built only from past transactions. Removing cross-period edges tests a different reliance on structure than removing edge attributes. A threshold chosen for F1 addresses a different decision from a fixed review budget. If a configuration exhausts memory, the experiment has established a feasibility boundary, not poor predictive performance.

This dependence is easy to lose in a leaderboard. Statements such as "graphs help" or "model A is best" omit the temporal horizon, visibility rule, graph construction, selection procedure, metric, and resource envelope under which the comparison was observed. Model-selection bias and data leakage can then enter through a split, a preprocessing step, or repeated inspection of test results [7]–[9]. The broader benchmarking literature has shown that task and protocol choices can change apparent progress [10]–[12]. Temporal graph benchmarks make several of these choices explicit [13]–[15], while graph-anomaly benchmarks broaden datasets and methods [16], [17]. Fraud screening still requires their concerns to be joined with graph construction, extreme prevalence, analyst capacity, and failed compute cells.

FraudShiftBench treats an evaluation result as a statement about a *deployment contract* $\Pi = (\mathcal{T}, \mathcal{V}, \mathcal{C}, \mathcal{S}, \mathcal{B}, \mathcal{R})$, whose components specify time, graph visibility, construction, model selection, decision budget, and resources. A result becomes claimable only through an evidence unit that records the prediction task, concrete contract, model and hyperparameters, seeds, metrics, prediction manifest, resource record, and integrity state. We then define a typed claim and a support relation between claims and evidence. This separation matters: insufficient evidence yields a blocked status; a compute failure yields resource-blocked; and an empirical predicate contradicted by complete evidence is refuted. None of those outcomes is silently converted into another.

The evidence does not reduce to one verdict about graphs. On Elliptic, strict versus isolated graph visibility reverses the top AUPRC ranking among MLP, GCN, and GraphSAGE. The same intervention leaves the MLP scores unchanged, improves GCN on Elliptic, and reduces GraphSAGE AUPRC on both real datasets. On synthetic IBM AML-Data, a strong tree baseline leads all eight baseline AUPRC cells, while the wider model and construction grid yields frequent AUPRC–fixed-threshold-F1 disagreement. Original transaction attributes, graph scale, recency, and degree capping have effects that depend on the cell. These results contradict both a universal graph benefit and a universal graph penalty. The measured resource boundary is equally specific: GINE is strongest by AUPRC in the four feasible Small graph-grid cells, but no Medium GINE score

exists under the recorded T4 envelope.

This paper makes four contributions.

1) It formalizes a fraud-specific deployment contract over six independent evaluation axes. The contract distinguishes temporal prediction, structural access, model selection, analyst capacity, and feasibility without treating one protocol as a universal default.
2) It defines a typed claim–evidence support relation and an executable validator. Scope expansion, missing predictions, incomplete seeds, integrity failure, and resource failure have different semantics; controlled mutations of locked artifacts test those transitions.
3) It provides a ten-seed empirical study across Elliptic, DGraphFin, and four IBM AML-Data Small/Medium regimes. The analysis isolates protocol, architecture, construction, metric, prevalence, scale, and runtime effects instead of pooling unlike prediction tasks into one ranking.
4) It releases an auditable benchmark artifact that connects manuscript numbers to predictions, locks, and deterministic generators. A saved-output GraphSafe-TTA analysis is retained as a bounded decision case study, with negative and non-significant comparisons reported alongside favorable ones.

The results address six questions: how graph-visibility protocols change conclusions (RQ1), whether those effects depend on architecture and dataset (RQ2), how IBM graph construction and scale alter rankings (RQ3), when rank and decision metrics disagree (RQ4), which performance–resource boundaries are observed (RQ5), and whether the support validator prevents unsupported promotion while preserving reproducibility (RQ6).

II. Related Work and Positioning

A. Graph-based fraud detection

Graph fraud research has largely advanced by changing the representation or learning rule. GraphConsis regularizes inconsistent neighborhoods [18]; CARE-GNN addresses camouflaged fraudsters through relation-aware aggregation [19]; and PC-GNN couples class-aware sampling with neighborhood aggregation for imbalance [20]. BWGNN targets anomalous graph signals in the spectral domain [21]. Directed-multigraph models preserve edge direction and multiplicity that homogeneous projections can erase [22], [23], while FraudGT develops an efficient transformer for financial graphs [24]. These methods sit on the foundations provided by GCN, GraphSAGE, graph attention, and graph isomorphism networks [25]–[28].

This modeling work explains why graph construction cannot be treated as neutral. A fraud detector may consume account links, transaction nodes, directed transaction edges, relation types, or aggregates computed from a historical subgraph. Two recent results are especially relevant. GADBench finds that simple non-GNN methods can be competitive across supervised graph-anomaly tasks [16]; the Graph Feature Preprocessor obtains efficient financial-crime models by composing graph-derived features with tabular learners [29]. We therefore use logistic regression and histogram gradient boosting as substantive baselines, and treat edge attributes, neighborhood summaries, degree caps, and recency windows as testable constructions. FraudShiftBench does not propose another universal detector. It asks which detector conclusion is supported under a stated information contract.

B. Financial and graph-anomaly benchmarks

Elliptic, DGraphFin, and IBM AML-Data cover markedly different tasks: illicit Bitcoin transaction nodes, anomalous account nodes, and laundering transaction edges [4]–[6]. Reviews of graph-based fraud detection already document the diversity of entity definitions, relations, and operational assumptions [30]. Broader anomaly benchmarks expand method and dataset coverage [16]; BAG adds dynamic graph tasks [17]; and GAD in the Wild studies scale, rarity, and missing attributes under deployment-oriented settings [31]. These works preclude a credible claim that ours is the first dynamic or realistic graph-anomaly benchmark.

The remaining gap is narrower. Existing benchmarks do not jointly type the temporal unit, graph visibility, construction, selection rule, investigation budget, and compute envelope of a fraud claim, then check that the necessary predictions and paired cells exist. Dataset breadth alone cannot resolve that gap. FraudShiftBench consequently keeps the three task units separate and reports conditional comparisons within each compatible scope.

C. Temporal evaluation and distribution shift

OGB established common graph tasks, evaluators, and dataset splits at scale [32]; GOOD isolates graph distribution shifts [33]. For dynamic link prediction, Poursafaei et al. show that evaluation choices such as negative sampling and inductive status materially affect conclusions [11]. TGB, TGB 2.0, and BenchTemp extend standardized evaluation across temporal, heterogeneous, and knowledge graphs, with attention to efficiency and temporal order [13]–[15].

Fraud detection adds an unusual separation between structural and label availability. An account or transaction may be visible when scored even though its eventual fraud label is not. The historical graph used to construct node features may also cover a different interval from the labels used to fit a classifier. Our deployment contract records those choices separately. It complements general temporal benchmarks by asking a fraud-specific question: which covariates, endpoints, edges, labels, and decision capacities are available at each stage?

D. Validity, documentation, and benchmark governance

Standard splits can conceal sampling assumptions [34], while feature leakage and repeated model selection can invalidate an otherwise reproducible pipeline [7]–[9]. The benchmark lottery describes a related external-validity problem: performance depends on which benchmark happens to stand in for the intended task [10]. BetterBench evaluates benchmark quality systematically rather than accepting a leaderboard at face value [12]; recent work also argues that broad state-of-the-art claims require commensurately broad evidence [35].

Documentation frameworks solve another part of the problem. Data Statements, Datasheets, Data Cards, and Model Cards make intended use, composition, and evaluation conditions visible [36]–[39]. BenchmarkCards and Eval Factsheets extend structured reporting to benchmarks and evaluations [40], [41]. Measurement work cautions that even well-documented metrics can fail to represent the construct of interest [42]. FraudShiftBench builds on these practices but changes the unit of validation: a typed empirical claim must be supported by complete evidence under its requested deployment scope. The validator does not certify scientific truth; it checks the declared relation among scope, predictions, pairing, integrity, and feasibility.

E. Ranking, calibration, and constrained decisions

Rare-event screening requires more than a single discrimination score. Precision–recall curves are informative under severe imbalance [1], but AUPRC still describes score ordering rather than the alert policy that investigators execute. Probability calibration can improve score interpretation [43], and prior-shift correction can adapt posterior probabilities when class prevalence changes [44]. Selective classification studies risk as a function of accepted coverage [45]; cost-sensitive learning attaches explicit loss to different errors [2]; and feedback-guided anomaly discovery models inspection of a ranked queue [3].

This motivates a strict distinction between *rank claims* and *decision claims*. AUROC and AUPRC depend on ordering. F1, precision or recall at a review budget, and cost-sensitive risk depend on a threshold or capacity. A strictly increasing recalibration preserves the former while it may change the latter. FraudShiftBench records the decision rule in the contract so that an operational gain cannot be described as a repaired ranking.

III. Deployment Contracts and Claim Scope

A. Prediction task and label eligibility

Let a temporal fraud task be $D = (\mathcal{I}, X, t, y, G_0, U)$, where $\mathcal{I}$ indexes candidate predictions, X contains observed attributes, t_i is event time, G_0 is the source relational record, and U identifies the prediction unit. The three units studied here are a transaction node, an account node, and a transaction edge. Metrics are comparable within a unit and contract; we do not pool their values into a cross-task leaderboard.

The repository label convention for Elliptic and DGraphFin is $y_i \in \{0, 1, 2\}$, with 0 denoting unknown or ineligible, 1 fraud, and 2 normal. Supervision is defined only on

$$m_i = \mathbf{1}[y_i \neq 0], \qquad z_i = \mathbf{1}[y_i = 1] \quad \text{for } m_i = 1. \tag{1}$$

Thus unknown labels never enter a train, validation, or test mask. IBM AML-Data is materialized directly as the binary edge label $z_i = \texttt{is\_laundering}_i$ and has no unknown class in the evaluated task. A fitted model produces a score $s_i \in \mathbb{R}$ for each eligible evaluation unit.

B. Deployment contract

A deployment contract is the six-tuple

$$\Pi = (\mathcal{T}, \mathcal{V}, \mathcal{C}, \mathcal{S}, \mathcal{B}, \mathcal{R}). \tag{2}$$

Its coordinates are recorded separately and are non-substitutable: holding one fixed does not make a change in another scientifically ignorable. They need not be statistically or causally independent.

- $\mathcal{T}$ fixes temporal order, train/validation/test intervals, and prediction horizon.
- $\mathcal{V}$ states which nodes and edges are visible when features are constructed, the model is fitted, and predictions are made. Label visibility is specified separately.
- $\mathcal{C}$ maps the source record G_0 to the evaluated graph and attributes. It includes the prediction unit, direction, edge features, degree limits, and historical window.
- $\mathcal{S}$ records the validation data, model-selection metric, stopping rule, and threshold-selection policy.
- $\mathcal{B}$ defines the decision surface: a threshold, review capacity K, selective coverage, or explicit error costs.
- $\mathcal{R}$ declares the hardware and software envelope, memory and disk guards, runtime measurement, and failure semantics.

Two contracts can share a chronological split while differing in graph access, or share a graph while answering different review-budget questions. Comparisons across contracts must name the components that changed.

Figure 1 summarizes the information flow. The contract fixes the scientific scope before a run becomes evidence; the support test then checks whether the requested claim is covered.

C. Evidence units and typed claims

A result file alone is not the evidence object. We define

$$e = (D, \nu, U, \Pi, C_G, M, H, \Sigma, Q, P, R_e, Z), \tag{3}$$

where ν is the dataset variant, C_G is the concrete graph materialization, M and H identify the method and hyperparameters, Σ is the seed set, and Q is the metric set. P points to prediction-level outputs, R_e records measured resource outcome, and Z contains provenance and integrity state. C_G is retained even though construction is part of Π: the contract states the scientific choice, while C_G identifies its realized artifact and parameters.

A claim is also typed:

$$c = (\Omega, q, \Delta, \mu, d, u, \iota). \tag{4}$$

Here Ω is the dataset/model/protocol scope, q is a quantifier, Δ names the comparison, μ is the metric, d is the asserted direction or ordering, u specifies uncertainty and multiplicity requirements, and ι gives the deployment interpretation. For example, "GraphSAGE loses AUPRC under isolated access on Elliptic" and "graphs are harmful" differ in both scope and quantifier even if they cite the same cell.

For a set of evidence units $\mathcal{E}$, we write $\mathcal{E} \models c$ only if all of the following hold: (i) every cell required by Ω and q is present under a compatible contract; (ii) Z passes the declared schema, provenance, and construct checks; (iii) P is present whenever

TABLE I
CLOSEST BENCHMARK FAMILIES AND THE DISTINCTIONS NEEDED HERE. "NO" MEANS THAT THE FEATURE IS NOT A STATED BENCHMARK AXIS; IT DOES NOT QUESTION THE RIGOR OF THE CITED WORK.

Work	Temporal	Graph contract	Capacity	Resources	Predictions	Claim support
GADBench	No	Limited	No	No	Benchmark	No
TGB / BenchTemp	Yes	Inductive tasks	No	Efficiency	Benchmark	No
BAG	Yes	Task/model breadth	No	Scale	Benchmark	No
GAD in the Wild	Partial	Scale/missing attrs.	No	Scale	Limited	No
BetterBench / Eval Fact-sheets	Generic	Documented	Documented	Documented	Metadata	Descriptive
FraudShiftBench	Yes	Visibility + construction	Top-K + cost	Typed status	Per-seed	Executable

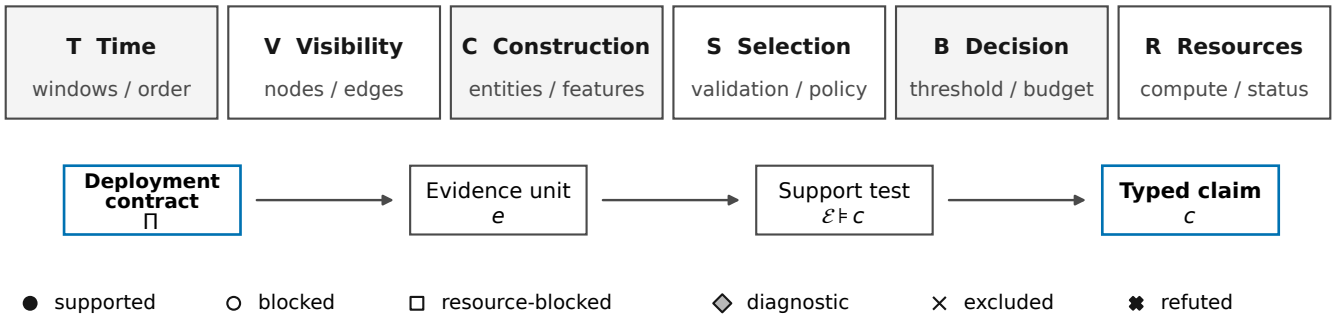

Fig. 1. FraudShiftBench information flow. Six non-substitutable coordinates define a deployment contract. A run becomes a typed evidence unit, and the support test admits only claims covered by its scope, predictions, uncertainty, integrity, and resource state. Blocked evidence has no implied performance value.

the claim requires prediction-derived metrics or reanalysis; (iv) the specified seeds and paired keys are complete; (v) the test, effect, interval, and multiplicity procedure in u can be executed; (vi) no requested predictive comparison crosses a resource-blocked cell; and (vii) the observed predicate d is true. The benchmark returns *supported*, *blocked*, *resource-blocked*, *diagnostic-only*, *excluded*, or *refuted in scope*. Blocked denotes inadequate evidence. Refuted denotes complete evidence that contradicts the predicate.

D. Useful consequences

The relation has four immediate properties.

Scope monotonicity. If c' widens the scope or quantifier of c, its required cell set contains that of c. Evidence sufficient for the narrower claim is not thereby sufficient for c'.

Evidence restriction. Removing a required seed, prediction manifest, or cell cannot upgrade an unsupported claim. This follows directly because each is a conjunct in the support test.

Protocol non-equivalence. Evidence under Π_1 may support a claim stated for Π_2 only when the components relevant to that claim coincide. Equal numerical values do not establish that two estimands are interchangeable.

Rank and resource separation. For scores without a change in tie order, a strictly increasing transform preserves the order used by AUROC and AUPRC, although a fixed threshold can yield different decisions [43]. A resource-blocked cell has no predictive score and therefore has no position in either ranking.

The validator implements these requirements over immutable inventory records. Its role is deliberately limited: it can detect an unsupported scope expansion or an absent prediction manifest, but it cannot establish construct validity or causal explanation by itself.

IV. BENCHMARK INSTANTIATION

A. Datasets and prediction units

Table II describes the evaluated tasks. Elliptic is a real, pseudonymized Bitcoin transaction graph with 203,769 transaction nodes, 234,355 source edges, and 49 time steps [4]. Its chronological labeled split contains 26,905/2,989/16,670 train/validation/test nodes; the test set has 1,083 illicit transactions. DGraphFin is a real anonymized account graph with 3,700,550 users and 4,300,999 directed source edges [5]. Incident-edge timestamps define 20 equal-count temporal buckets, with 947,462/107,932/170,207 eligible accounts in the three splits and 1,863 test positives.

IBM AML-Data contains synthetic directed transactions generated under higher- and lower-illicit-ratio regimes [6]. We evaluate Small and Medium variants. They range from 5.08 to 31.90 million transaction edges and from 515,088 to 2,077,023 account nodes. Test prevalence ranges from 0.058% to 0.177%, depending on variant and temporal window. Each transaction has eight attributes: log paid and received amounts, coded payment and receiving currencies, payment format, hour, weekday, and month. Eight account-history features record in/out counts, amount sums, and means. Large variants are retained in the contract as resource-blocked cells with no performance values.

These datasets do not define a common supervised object. Elliptic predicts a transaction-node label, DGraphFin an account-node label, and IBM a transaction-edge label. Their priors,

graph semantics, and candidate sets differ. We use them to test whether a qualitative claim survives across task contracts, not to crown a pooled winner.

B. Instantiated temporal and visibility contracts

Table III separates label windows, label-free history, graph visibility, model selection, decision policy, and resources. None of these coordinates substitutes for another.

Elliptic and DGraphFin use fixed chronological masks under three structural views. In *strict-inductive* evaluation, the model is fitted on the training subgraph; held-out structure becomes available only for evaluation and held-out labels remain hidden. *Isolated-inductive* uses the same label masks but removes cross-split edges from the held-out evaluation components. This intervention isolates reliance on connections across temporal partitions. In the *transductive* diagnostic, full graph structure is visible while validation and test labels remain hidden. Strict versus isolated is the main matched contrast; transductive results are not treated as a deployment default.

IBM uses stable chronological sorting under two label partitions. The late-window holdout assigns 60/20/20% of transactions to train, validation, and test. Early-to-late transfer uses 50/20/30%. Both protocols reuse one account-history matrix constructed from the first 60% of transactions and no fraud labels. For late-window holdout, that interval equals the classifier's training period. For early-to-late transfer, it includes label-free covariates from the 50–60% interval, which is the first half of its validation window. The early-to-late contract is therefore a 50% label-training protocol with 60% historical-covariate visibility, not a first-50%-only inductive graph. We expose this distinction because collapsing covariate and label availability would state a stronger protocol than the implementation supports.

C. Models and graph constructions

The real-graph protocol grid compares an MLP negative control with GCN [25] and mean-aggregating GraphSAGE [26]. For node representation $h_v^{(\ell)}$, the latter computes

$$h_v^{(\ell+1)} = \sigma\Big(W^{(\ell)}[h_v^{(\ell)} \,\|\, \mathrm{mean}_{u\in\mathcal{N}_\Pi(v)}\, h_u^{(\ell)}]\Big), \qquad (5)$$

where $\mathcal{N}_\Pi(v)$ changes with graph visibility. The MLP omits that term, so its strict–isolated difference is a check that the intervention touches graph access rather than the feature or label masks.

The IBM baseline grid includes balanced logistic regression, histogram gradient boosting [46], and a one-hop GraphSAGE-derived edge classifier. The latter concatenates each endpoint's eight history features with the mean history vector of its training-neighborhood, then appends the transaction attributes before a two-layer edge head. The stronger reference uses the same computation at width 64. It is an edge classifier over fixed one-hop summaries, not an unqualified implementation of full-neighborhood end-to-end GraphSAGE.

Matched controls modify one part of this reference. NoEdge zeros the eight transaction attributes. ShuffledEdge permutes them within each temporal split, preserving their marginal values but removing edge-level alignment. DegreeOnly replaces them with log endpoint in/out degrees. DegreeCap removes training edges incident to accounts above the 0.995 active-degree quantile. RecentWindow retains the most recent 50% of training edges. A one-layer edge-conditioned GIN variant (GINE) [47] instead learns node messages conditioned on edge attributes and scores edges from the two learned endpoint embeddings plus their transaction attributes. The account–account sender–receiver row merely records the already materialized directed transaction projection; it is an implementation alias, not another construction or independent result.

The supplement gives separate model, construction, and optimization cards at readable scale. The main paper retains the definitions needed to interpret the interventions but does not reproduce the full configuration inventory.

Small and Medium results use ten seeds per completed cell. The declared resource envelope includes preprocessing limits, prediction-export disk limits, measured runtime, and GPU failures. Medium GINE and the fixed DGraphFin GAT configuration exhausted Tesla T4 memory; the IBM Large lanes were stopped by a safe resource guard. These cells remain visible as feasibility outcomes and are excluded from performance ranks. A reduced DGraphFin GAT diagnostic does not fill the missing fixed-configuration cell.

V. Experimental Design and Statistical Methodology

The experiments answer six questions. RQ1 asks whether changing graph visibility changes a temporal fraud result. RQ2 asks whether that change is consistent across architectures and datasets. RQ3 studies graph construction, scale, and class-prior regime on IBM AML-Data. RQ4 compares ranking metrics with fixed-threshold and review-budget decisions. RQ5 treats runtime and memory feasibility as part of the deployment contract. RQ6 tests whether the support relation rejects incomplete or incorrectly widened claims. A saved-output GraphSafe analysis is retained as a bounded case study, not as a seventh method leaderboard.

A. Experiment Families and Controls

The visibility experiment is a fully crossed grid over Elliptic and DGraphFin; MLP, GCN, and mean-aggregating GraphSAGE; strict-inductive, isolated-inductive, and transductive evaluation; and seeds 1–10. The primary intervention compares strict with isolated visibility because their temporal masks and training graph are identical. The feature-only MLP is the negative control: a graph-only intervention should leave its scores unchanged. Transductive rows are useful context, but they are not substituted for either inductive contract.

The IBM study has two parts. A baseline grid compares balanced logistic regression, histogram gradient boosting, and a GraphSAGE-derived edge classifier with 32 hidden units. A matched graph grid uses the edge-aware h64 classifier as its reference and varies original edge attributes, zeroed attributes, shuffled attributes, degree-only features, degree capping, recent-history construction, and GINE where feasible. Each measured cell has ten seeds. We rank methods only within the same

TABLE II
DATASET AND TASK CARDS. SCALE IS NODES/ACCOUNTS FOLLOWED BY SOURCE EDGES/TRANSACTIONS. THE PREDICTION UNITS, TIME REPRESENTATIONS, PREVALENCES, AND EMPIRICAL SCOPES DIFFER, SO VALUES ARE NOT POOLED INTO ONE LEADERBOARD.

Dataset	Unit	Scale	Time / split	Test prior	Features	Protocols	Empirical scope
Elliptic	transaction node	203,769 / 234,355	49 steps; 30/4/15	6.497%	165 node	strict; isolated; transductive	real; node task
DGraphFin	account node	3,700,550 / 4,300,999	20 buckets; 14/2/4	1.095%	17 node	strict; isolated; transductive	real; node task
IBM HI-SMALL	transaction edge	515,088 / 5,078,345	50/20/30; 60/20/20	0.152–0.177%	8 node + 8 edge	early-to-late; late holdout	synthetic; Small/Medium
IBM HI-MEDIUM	transaction edge	2,077,023 / 31,898,238	50/20/30; 60/20/20	0.148–0.167%	8 node + 8 edge	early-to-late; late holdout	synthetic; Small/Medium
IBM LI-SMALL	transaction edge	705,907 / 6,924,049	50/20/30; 60/20/20	0.064–0.067%	8 node + 8 edge	early-to-late; late holdout	synthetic; Small/Medium
IBM LI-MEDIUM	transaction edge	2,032,095 / 31,251,483	50/20/30; 60/20/20	0.058–0.061%	8 node + 8 edge	early-to-late; late holdout	synthetic; Small/Medium

TABLE III
INDEPENDENT DEPLOYMENT-CONTRACT COORDINATES. IBM EARLY-TO-LATE USES LABELS FROM THE FIRST 50% BUT THE SHARED LABEL-FREE ACCOUNT-HISTORY MAP FROM THE FIRST 60%; IT IS NOT A FIRST-50%-ONLY FEATURE CONTRACT.

Protocol	Label windows	Label-free history	Graph visibility	Selection	Decision	Resource envelope
Strict inductive	chronological	train period	held-out structure only at evaluation	validation F1	argmax / rank	recorded CPU/CUDA
Isolated inductive	same masks	train period	remove train–held-out edges	validation F1	argmax / rank	same as strict
Transductive	same masks	full label-free graph	full graph; labels hidden	validation F1	diagnostic	recorded CPU/CUDA
IBM early-to-late	50/20/30 labels	first 60% covariates	shared 60% account history	fixed schedule	$\tau = 0.5$ / budgets	CPU + T4 guards
IBM late holdout	60/20/20 labels	first 60% covariates	training-period account history	fixed schedule	$\tau = 0.5$ / budgets	CPU + T4 guards

variant, temporal protocol, and feasible configuration set. The account–account sender–receiver row is an implementation alias of the reference construction, so it is an equivalence check rather than an independent replication.

B. Optimization, Selection, and Compute

The Elliptic and DGraphFin neural models use full-graph forward passes, class-weighted cross-entropy, AdamW with learning rate 10^{-3} and weight decay 5×10^{-4}, cosine decay, gradient clipping at 1, and dropout 0.5. Elliptic uses three 256-unit layers; DGraphFin uses two 64-unit layers. Training is capped at 200 epochs. Validation F1 is evaluated at epoch 1 and every ten epochs. The configured patience is 40 validation checks, which cannot be reached within this cap; the procedure therefore selects the best scheduled validation checkpoint rather than stopping early. Repository labels are three-coded, but loss and evaluation exclude unknown nodes; illicit/fraud is the positive class.

For IBM AML-Data, balanced logistic regression uses standardized transaction features and at most 500 iterations. Histogram gradient boosting uses 160 iterations and learning rate 0.06. The h32 graph classifier is trained for 20 epochs with edge batches of 65,536. The h64 reference, its matched controls, and GINE use 30 epochs and batches of 32,768. These graph models use AdamW with learning rate 10^{-3}, weight decay 10^{-4}, positive-weighted binary cross-entropy, and dropout 0.1. Saved IBM F1 uses a fixed threshold of 0.5; it is therefore an operating-point result, not a validation-optimized upper bound.

The imported commands record CUDA execution for the two real-data grid and configuration-specific elapsed time for IBM. IBM feasibility and out-of-memory records are tied to a single Tesla T4 envelope. A legacy path token containing "P100" is an alias: its runtime record identifies CUDA device 0 and the validation layer normalizes it to the T4 lane. We do not infer hardware from path names. Runtime comparisons are made only inside a common experiment family and matching variant/protocol cell.

C. Metrics and Operational Quantities

AUROC measures positive–negative ordering; average precision (AUPRC) summarizes the precision–recall ranking and is emphasized for the highly imbalanced tasks [1]. For context, the supplement reports $\mathrm{AUPRC}/\pi_{\mathrm{test}}$, where π_{test} is test prevalence. This ratio is descriptive lift over the prevalence baseline, not a prevalence-invariant replacement metric. F1 is the harmonic mean of precision and recall at the family-specific threshold described above.

For a review fraction $b \in \{0.005, 0.01, 0.02\}$, $K = \lceil bN \rceil$ and Precision@K and Recall@K are computed after a stable descending score sort. The saved-output analysis also reports

$$R_{1,5} = \frac{\mathrm{FP} + 5\,\mathrm{FN}}{N}, \tag{6}$$

which encodes one declared false-positive cost and a fivefold false-negative cost. Other cost ratios are sensitivity analyses, not claims about a bank's actual loss function. Expected calibration error and Brier score diagnose probability quality; worst-block regret compares a policy with the hindsight-best saved branch within each time block. These quantities proxy screening behavior. They do not establish downstream investigative value.

D. Uncertainty and Multiple Comparisons

All principal summaries use seeds 1–10. We report the arithmetic mean and sample standard deviation. Confidence intervals are deterministic two-sided 95% percentile intervals from 10,000 bootstrap resamples with a fixed analysis seed [48]. Aligned comparisons use the two-sided Wilcoxon signed-rank test and paired Cohen's d_z; zero differences return $p = 1$. Holm's step-down procedure controls family-wise error [49].

We declare the correction families used in the rebuilt analysis. For the visibility grid, Holm correction is applied separately for AUPRC, AUROC, and F1, each over the six dataset–model strict-versus-isolated comparisons. IBM construction effects pair candidate and reference on variant, protocol, and seed. The four variant–protocol contexts are fixed benchmark conditions rather than 40 exchangeable units, so we average their paired differences within seed before inference. A complete size therefore has ten inferential seed blocks from 40 raw matched rows; the supplement also reports every context-specific ten-seed effect. Small and Medium are separated, and Holm correction is applied within each size–metric family. GINE contributes Small pairs only. Rank correlations across feasible configuration means are descriptive.

GraphSafe inference uses ten seed blocks. Within each seed, the six protocol–model contexts are averaged first; the inferential sample is therefore $n = 10$, not 60. The declared family contains four saved-output methods versus simple averaging, two datasets, and six metrics, for 48 paired comparisons. Holm correction spans all 48. Blocked cells have no imputed score, receive no rank, and are absent from paired and Pareto sets.

VI. Protocol and Architecture Effects

A. RQ1: Does Graph Visibility Change the Leaderboard?

On Elliptic, it changes the leading model in the evaluated three-model set. Under strict-inductive visibility, GraphSAGE has mean AUPRC 0.6114, ahead of GCN (0.4666) and MLP (0.4522). Under isolated-inductive visibility, GCN leads at 0.5605, followed by GraphSAGE at 0.4797 and the unchanged MLP at 0.4522. Thus the top-ranked model switches from GraphSAGE to GCN without changing temporal masks, training data, seed set, or selection metric.

DGraphFin does not show the same top-rank switch. GraphSAGE remains first, but its mean AUPRC falls from 0.0227 to 0.0178. MLP remains at 0.0140, while GCN changes from 0.0116 to 0.0118. A protocol can therefore alter the margin or the winner; the Elliptic result is not evidence that every graph-fraud leaderboard must reverse.

B. RQ2: Are Visibility Effects Architecture-Dependent?

They are. GraphSAGE loses 21.5% of strict AUPRC on both datasets. The paired absolute changes are -0.1317 on Elliptic (95% CI $[-0.1422, -0.1211]$, $d_z = -7.40$, Holm $p = 0.012$) and -0.0049 on DGraphFin (95% CI $[-0.0064, -0.0033]$, $d_z = -1.88$, Holm $p = 0.016$). The similar relative percentage masks a large difference in absolute task scale.

GCN moves in the opposite direction on Elliptic: $+0.0938$ AUPRC, or $+20.1\%$ (95% CI $[0.0790, 0.1082]$, $d_z = 3.77$, Holm $p = 0.012$). Its DGraphFin change is $+0.0002$ with an interval crossing zero and Holm $p = 1$. MLP is numerically identical under the two graph views on both datasets, as expected from the intervention.

These patterns reject a uniform "graph structure helps" or "graph structure hurts" account. Isolating held-out nodes removes train-to-held-out message paths while keeping features, labels, and masks fixed. The result measures sensitivity to that visibility contract. It does not by itself identify homophily, oversmoothing, degree shift, or another mechanism as the cause. For model selection, the practical point is simpler: a validation winner obtained with one graph view is not automatically the winner for another deployment view.

VII. Construction, Metric, and Resource Effects

A. RQ3: What Changes Across IBM Constructions and Scale?

Table V and Fig. 3 answer the baseline question first. Histogram gradient boosting has the highest mean AUPRC in all eight HI/LI, Small/Medium, early/late contexts. This result is confined to the three recorded baseline families and does not imply that trees dominate AML generally.

The graph-construction grid answers a different question. GINE h64 has the highest mean AUPRC in all four feasible Small contexts, whereas Medium GINE has no performance observation. Table VI identifies the best measured non-reference construction by AUPRC in each cell and reports that same construction's matched changes on all three metrics. It is not a multi-metric winner table.

Matched controls show that transaction attributes carry ranking information for the h64 classifier. Zeroing edge features lowers AUPRC relative to the reference by 0.00267 on Small (seed-block CI $[-0.00424, -0.00109]$, Holm $p = 0.0820$) and 0.01206 on Medium (CI $[-0.01395, -0.01022]$, Holm $p = 0.0117$). The Small corrected test is not significant, although all four context means are negative. Shuffling those features lowers AUPRC by 0.00330 on Small (Holm $p = 0.0137$) and 0.01208 on Medium ($p = 0.0117$); again, all context means are negative. These interventions preserve either the graph or feature marginals, so they support a bounded alignment result for this classifier, strongest on Medium and for the shuffled control. They do not prove that the attributes causally improve every fraud model.

Construction effects also change with scale and metric. Degree capping raises Small mean F1 by 0.00473 and AUROC by 0.02354, but none of its Small AUPRC, AUROC, or F1 tests survives Holm correction. On Medium it raises F1 by 0.00587 (Holm $p = 0.0352$) while reducing AUPRC by 0.02086 ($p = 0.0117$). The recent-window graph shows a different Medium tradeoff: AUROC rises by 0.02525 ($p = 0.0117$) and F1 by 0.00419 ($p = 0.0352$), while AUPRC falls by 0.01324 ($p = 0.0117$). Figure 4 places all three matched metric effects on one scale. These are fixed-grid empirical interactions, not identified mechanisms; truncating history changes information, degrees, and computation at once.

B. RQ4: When Do Ranking and Decision Metrics Disagree?

AUPRC and F1 select different winners in 11 of 16 feasible variant–protocol contexts: three of eight baseline-grid contexts

TABLE IV
PAIRED AUPRC EFFECTS OF STRICT VERSUS ISOLATED GRAPH VISIBILITY OVER TEN SHARED SEEDS. Δ IS ISOLATED MINUS STRICT; INTERVALS ARE DETERMINISTIC 95% BOOTSTRAP INTERVALS. HOLM CORRECTION COVERS THE SIX AUPRC COMPARISONS.

Dataset	Model	Strict	Isolated	Δ	95% CI	Rel. Δ (%)	d_z	Holm p
DGraphFin	GCN	0.0116	0.0118	+0.0002	[-0.0013, +0.0019]	+1.6	+0.07	1.000
DGraphFin	MLP	0.0140	0.0140	+0.0000	[+0.0000, +0.0000]	+0.0	+0.00	1.000
DGraphFin	GraphSAGE	0.0227	0.0178	-0.0049	[-0.0064, -0.0033]	-21.5	-1.88	0.016
Elliptic	GCN	0.4666	0.5605	+0.0938	[+0.0790, +0.1082]	+20.1	+3.77	0.012
Elliptic	MLP	0.4522	0.4522	+0.0000	[+0.0000, +0.0000]	+0.0	+0.00	1.000
Elliptic	GraphSAGE	0.6114	0.4797	-0.1317	[-0.1422, -0.1211]	-21.5	-7.40	0.012

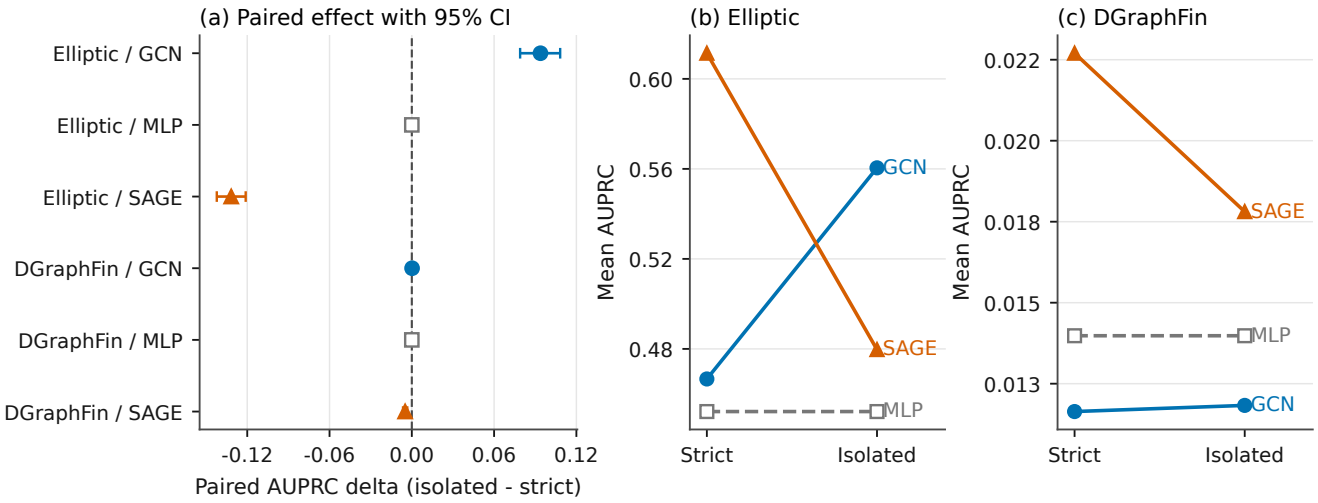

Fig. 2. Strict-to-isolated AUPRC effects over ten paired seeds. Horizontal intervals are deterministic 95% bootstrap intervals. The MLP control is unchanged; graph-model direction and magnitude depend on architecture and dataset.

TABLE V
IBM BASELINE AUPRC MEANS OVER TEN RUNS. BOLD MARKS THE BEST OF THE THREE RECORDED FAMILIES; FIG. 3 GIVES UNCERTAINTY.

Cell	HistGB	LR	SAGE	Best	Gap
HI-SMALL / Early	**0.0657**	0.0144	0.0038	HistGB	0.0514
HI-SMALL / Late	**0.0964**	0.0173	0.0071	HistGB	0.0792
HI-MEDIUM / Early	**0.0890**	0.0139	0.0248	HistGB	0.0641
HI-MEDIUM / Late	**0.1693**	0.0157	0.0371	HistGB	0.1322
LI-SMALL / Early	**0.0126**	0.0039	0.0021	HistGB	0.0087
LI-SMALL / Late	**0.0171**	0.0040	0.0022	HistGB	0.0130
LI-MEDIUM / Early	**0.0143**	0.0037	0.0069	HistGB	0.0073
LI-MEDIUM / Late	**0.0297**	0.0040	0.0078	HistGB	0.0219

and all eight graph-grid contexts. This count is specific to the saved F1 threshold of 0.5; it is not a threshold-invariant decision result. Figure 5 shows the clearest example. In HI-Medium late-window evaluation, the h64 reference ranks first of six graph configurations by AUPRC (0.0500) but sixth by F1 (0.0127). Degree capping ranks sixth by AUPRC (0.0127) and first by F1 (0.0221). RecentWindow ranks first by AUROC (0.9333), fifth by AUPRC, and second by F1.

The GINE result makes the distinction concrete. Against the h64 reference on the four Small contexts, GINE raises mean AUPRC by 0.01190 (seed-block CI $[0.00934, 0.01447]$, $d_z = 2.70$, Holm $p = 0.0137$); every context mean is positive. Its mean F1 is lower by 0.00206 (CI $[-0.00371, -0.00052]$), but that dependence-aware corrected test is not significant ($p = 0.252$). A high AUROC can likewise coexist with modest precision in a rare-positive task. Monotone recalibration may change fixed-threshold decisions, but it preserves score order and therefore cannot repair an AUPRC or AUROC ranking [43], [44].

C. RQ5: What Does the Resource Contract Exclude?

Resource cost changes the admissible comparison set. Averaged over the four Small contexts, GINE takes 449.47 seconds versus 49.31 seconds for the h64 reference, about nine times longer. On Medium, RecentWindow averages 141.65 seconds versus 275.19 seconds for the reference, while exchanging lower AUPRC for higher AUROC and F1. Figure 6 marks Pareto status only within a common variant and protocol; it does not merge runtimes from different tasks or hardware records. Table VII states how the unmeasured cells enter the benchmark.

The two Medium GINE lanes each exhausted Tesla T4 memory before producing any of 20 planned outputs. HI-Large and LI-Large were stopped by the safe resource guard and likewise have zero results and predictions. The fixed DGraphFin GAT h64/l2 cell is also T4-OOM; an h32/l1 diagnostic cannot fill that cell because it changes the model. These statuses

TABLE VI
BEST MEASURED NON-REFERENCE IBM CONSTRUCTION BY AUPRC IN EACH CELL. DELTAS COMPARE THAT SAME CONSTRUCTION WITH H64 ON AUPRC, AUROC, AND F1. THE HOLM VALUE IS THE DEPENDENCE-AWARE SIZE-LEVEL AUPRC TEST OVER TEN SEED BLOCKS, NOT A CONTEXT-SPECIFIC TEST.

Cell	Ref AUPRC	Best construction	ΔAUPRC	ΔAUROC	ΔF1	Holm p	Feasibility
HI-SMALL / Early	0.0103	GINE	+0.0158	+0.0290	-0.0031	0.014	Small measured
HI-SMALL / Late	0.0110	GINE	+0.0203	+0.0354	-0.0020	0.014	Small measured
HI-MEDIUM / Early	0.0424	Degree	-0.0172	-0.1130	-0.0008	0.012	GINE Medium blocked
HI-MEDIUM / Late	0.0500	Degree	-0.0174	-0.1170	+0.0054	0.012	GINE Medium blocked
LI-SMALL / Early	0.0032	GINE	+0.0066	-0.0028	-0.0012	0.014	Small measured
LI-SMALL / Late	0.0057	GINE	+0.0049	-0.0180	-0.0020	0.014	Small measured
LI-MEDIUM / Early	0.0103	Recent	-0.0037	+0.0312	+0.0007	0.012	GINE Medium blocked
LI-MEDIUM / Late	0.0111	Degree	-0.0033	-0.0440	-0.0025	0.012	GINE Medium blocked

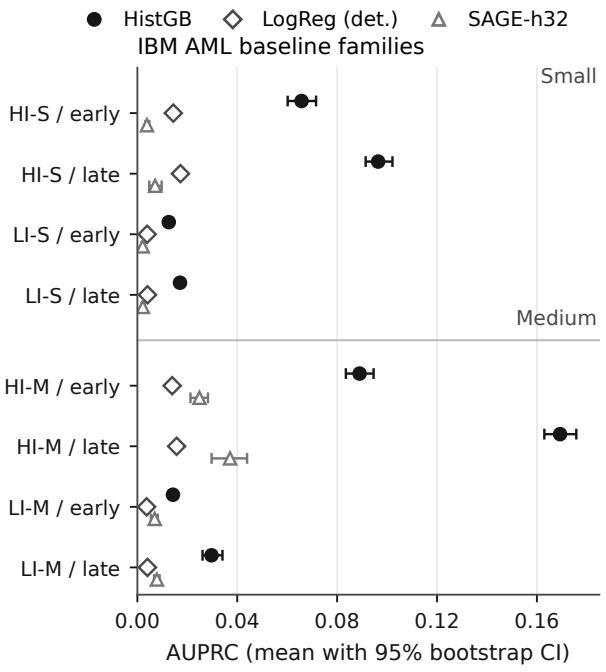

Fig. 3. IBM baseline-family AUPRC. Points are ten-run means and whiskers are deterministic 95% bootstrap intervals. Deterministic LR cells retain zero-width intervals.

answer feasibility questions under recorded envelopes. They say nothing about predictive performance on larger hardware.

VIII. FRAMEWORK VALIDATION AND A BOUNDED DECISION CASE

A. RQ6: Does the Support Relation Prevent False Promotion?

We exercised the validator with 14 in-memory claim mutations and evidence ablations. All 14 produced the expected status transition. A complete Elliptic visibility claim remained supported. Removing a required cell, a seed, or a prediction-manifest reference yielded incomplete-scope, incomplete-seed, and missing-prediction blocks. Widening Small GINE to Medium or IBM Small/Medium to Large produced a resource block rather than a performance conclusion. Integrity-failed imports, a non-independent construction alias, and construct-invalid metadata were excluded. Universal graph-harm and universal GraphSafe-improvement claims were marked refuted because observed cells contradict their quantifiers.

TABLE VII
UNMEASURED RESOURCE CELLS. STATUS IS A FEASIBILITY OUTCOME UNDER THE DECLARED ENVELOPE, NEVER A PREDICTIVE SCORE.

Cell	Envelope	Status	Benchmark treatment
IBM AML HI-Large	safe resource guard	GUARD-BLOCKED	excluded from predictive ranks, matched means, and Pareto sets
IBM AML LI-Large	safe resource guard	GUARD-BLOCKED	excluded from predictive ranks, matched means, and Pareto sets
IBM AML HI-Medium GINE h64	single Tesla T4 CUDA OOM	T4-OOM	excluded from predictive ranks, matched means, and Pareto sets
IBM AML LI-Medium GINE h64	single Tesla T4 CUDA OOM	T4-OOM	excluded from predictive ranks, matched means, and Pareto sets
DGraphFin GAT h64/l2	Tesla T4 CUDA OOM	T4-OOM	excluded from predictive ranks, matched means, and Pareto sets
DGraphFin GraphSAGE max-pool rerun	larger GPU required	UNMEASURED	excluded from predictive ranks, matched means, and Pareto sets

Figure 7 compares expected and observed outcomes across the complete controlled suite.

The most consequential construct failure came from the temporal-stress package. Its runner iterates over three names, but never passes the stress argument to the benchmark harness. For each of 120 dataset–protocol–model–seed base cells, all seven performance metrics are exactly identical under the three labels. The nominal 360 rows therefore contain 120 scientific cells and 240 metadata-labeled duplicates, not three temporal conditions. We exclude the labels from stress conclusions and retain at most one deduplicated representative per base cell as supplementary rerun evidence.

A filename/count-only audit would also promote 38 integrity-failed or noncomparable result files, 20 memory-reduced GAT diagnostics, and 12 hardware-alias rows incorrectly. Across the audited families, 310 result files and 310 prediction files are at risk of such misclassification. Deterministic regeneration provides a separate check: rerunning the analysis left all 28 generated-analysis CSV hashes and all eight audited canonical input hashes unchanged. These tests establish validator behavior and reproducible derivation, not external validity or causal correctness.

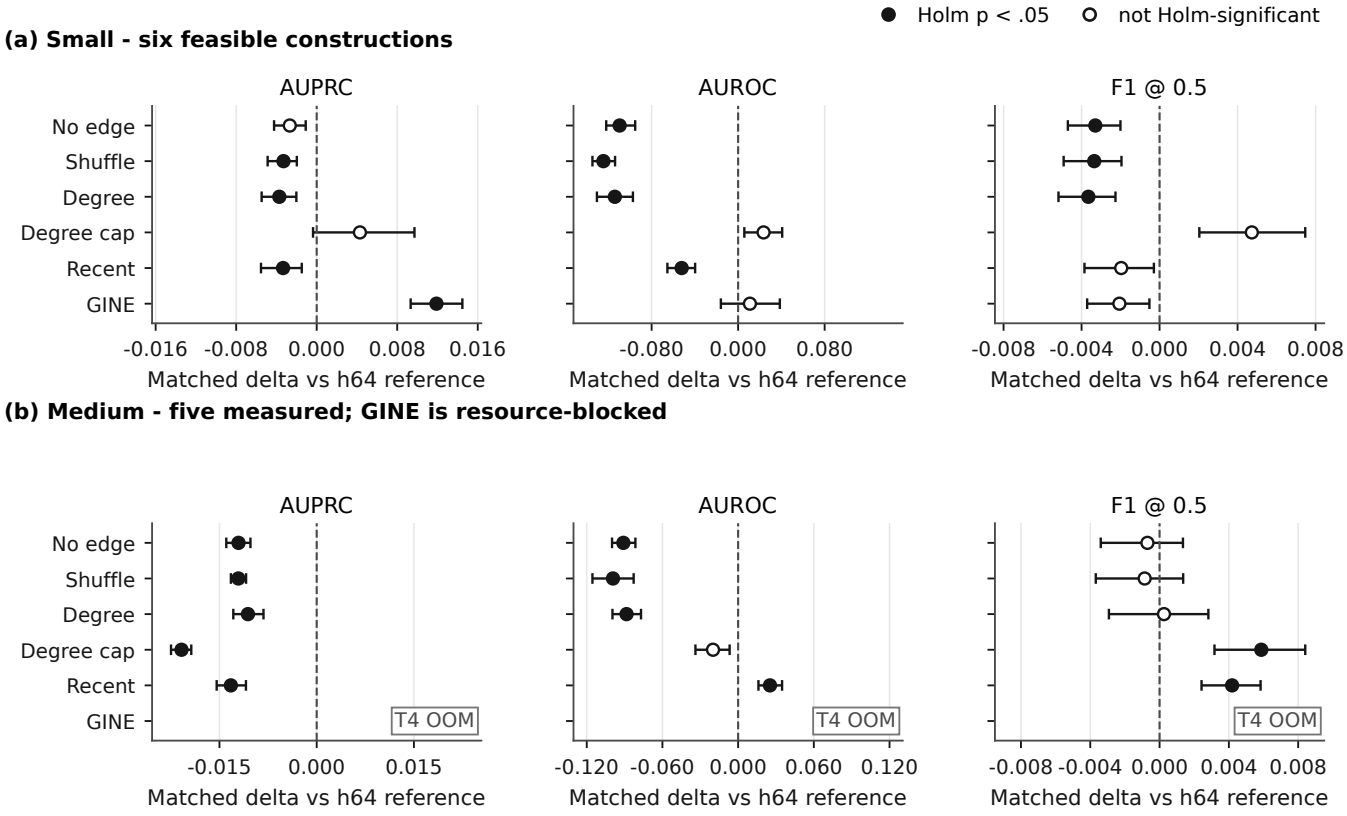

Fig. 4. Candidate-minus-reference IBM effects for AUPRC, AUROC, and F1. The four fixed contexts are averaged within seed before interval estimation and testing ($n = 10$ seed blocks). GINE is Small-only; a construction can help one metric while reducing another.

TABLE VIII
BOUNDED SAVED-SCORE COMPARISON OVER TEN SEED BLOCKS. LOWER ILLUSTRATIVE COST RISK IS BETTER; NO GRAPHSAFE CONTRAST SURVIVES HOLM CORRECTION OVER THE DECLARED 48-TEST FAMILY.

Data	Method	F1	R@1%	Risk
DGraphFin	Best val	0.036	0.014	0.473
DGraphFin	GraphSafe	0.043	0.021	0.291
DGraphFin	Average	0.036	0.014	0.452
Elliptic	Best val	0.532	0.120	0.180
Elliptic	GraphSafe	0.533	0.118	0.178
Elliptic	Average	0.551	0.128	0.171

B. GraphSafe as a Worked Decision-Level Example

GraphSafe combines saved graph and feature-model scores using quantities fit on training/validation data only. Its reliability risk aggregates score disagreement, rank disagreement, and entropy. A validation-selected cutoff chooses the graph score in low-risk cases and the feature score otherwise; temperatures, score thresholds, and the conservative policy gate are also set without test labels. The conservative policy can retain the validation-best branch when its reliability rules veto an override. Simple averaging and the validation-best branch are the principal comparators. The analysis uses saved predictions and performs no retraining.

On DGraphFin, conservative GraphSafe has favorable descriptive means relative to simple averaging: F1 0.0426 versus 0.0365, Recall@1% 0.0209 versus 0.0143, and risk 0.2913 versus 0.4522. After averaging the six protocol–model contexts within each seed, the paired improvements are 0.00610 F1 (95% CI $[0.00370, 0.00868]$), 0.00662 Recall@1% (95% CI $[0.00402, 0.00944]$), and 0.16092 lower risk (95% CI $[0.06409, 0.28513]$). Their Holm-adjusted p-values in the declared 48-comparison family are 0.109, 0.0938, and 0.225.

Elliptic provides the necessary negative comparison. Simple averaging has higher mean F1 (0.5513 versus 0.5333), higher Recall@1% (0.1276 versus 0.1176), and lower risk (0.1709 versus 0.1783). The seed-blocked differences favor simple averaging, with adjusted p between 0.0938 and 0.109 for these outcomes. No one of the 48 GraphSafe-versus-average tests has Holm-adjusted $p < 0.05$.

GraphSafe is therefore useful here as a worked example of claim discipline. Its DGraphFin aggregate is promising under a declared cost and review budget; its Elliptic aggregate is worse than a cheap ensemble; and the corrected family does not support universal dominance. Temperature or prior correction should not be described as a rank repair, and the policy has not been validated as an automated AML decision rule.

IX. IMPLICATIONS, REPRODUCIBILITY, AND VALIDITY

A. Implications for Fraud Benchmarking

The results favor contract-indexed reporting over a pooled leaderboard. A model name and dataset name do not identify

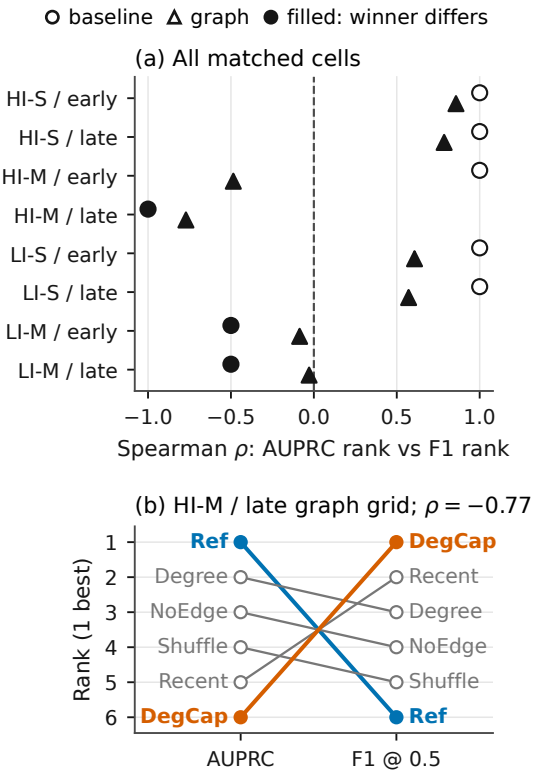

Fig. 5. AUPRC–F1 rank association and the HI-Medium late-window reversal. F1 uses the saved 0.5 threshold; AUPRC integrates score ordering.

a result unless temporal masks, graph visibility, construction, selection rule, decision budget, and resource envelope are also fixed. This matters even in a small model grid: Elliptic changes its AUPRC leader under a graph-only intervention, while DGraphFin keeps the leader but changes its margin. On IBM AML-Data, AUPRC and F1 choose different configurations in 11 of 16 feasible contexts.

Three reporting rules follow. First, graph visibility should be treated as an experimental variable, with a feature-only control when possible. Second, ranking, thresholded decisions, review capacity, and calibration should be reported as distinct claim types. Third, an out-of-memory cell belongs in a feasibility table, not at the bottom of a performance table. These rules do not prescribe one deployment contract; they make the chosen contract inspectable.

Cross-dataset aggregation would be especially misleading here. Elliptic predicts illicit transaction nodes, DGraphFin predicts fraudulent account nodes, and IBM AML-Data predicts laundering transaction edges. Their positive rates and temporal units differ, and IBM is synthetic. We therefore use each dataset to test a scoped proposition rather than average them into a single benchmark score.

B. Artifact and Reproduction Paths

The artifact connects each paper number to a source file, filter, seed set, aggregation rule, evidence lock, and output location. It includes dataset/protocol cards, a typed claim ledger, prediction-manifest references, resource and exclusion records, deterministic analysis code, figure/table builders, and the support validator. This extends documentation practices such as datasheets, model cards, and benchmark cards by making the claim-to-evidence relation executable [37], [39], [40].

The primary IBM package contains 840 validated result files and 840 corresponding prediction exports across the measured Small/Medium cells. That count describes provenance coverage, not 840 independent configurations. The real-data visibility family contains 180 result rows and 180 prediction files. Canonical locks distinguish these from partial imports, compatibility aliases, diagnostics, and resource-blocked plans.

Two reproduction paths are provided. The no-training path validates locks and manifests, recomputes all statistics from saved results and predictions, rebuilds tables and figures, executes the support mutations, and compiles both documents. The full path reacquires each dataset under its own access terms and reruns the training families on suitable compute. Raw restricted datasets, credentials, local absolute paths, failed temporary archives, and platform metadata are excluded from the release. The package claims no external artifact badge.

C. Threats to Validity

Internal validity. The strict and isolated real-data protocols share training code and masks, but a software error could still alter visibility in an unintended way. The MLP invariance check, prediction manifests, paired seeds, and graph-mode records reduce this risk. Selection differs across experiment families: the real-data grid saves the best validation-F1 checkpoint, IBM uses fixed training schedules and a 0.5 threshold, and GraphSafe fits its gate on validation reliability. Comparisons are confined to a family for this reason.

The IBM early-to-late classifier uses labels from the first 50% of transactions, but its shared node-history matrix is built from the first 60%. Thus node features include label-free covariates from the 50–60% interval, which is the first half of that protocol's validation window. This is transductive covariate access, not test-label leakage. We state it as part of the contract and do not call the early-to-late features "first-50%-only." The temporal-stress duplicate-label defect demonstrates why manifest completeness alone is insufficient; construct arguments must reach the executed harness.

Construct validity. AUPRC measures score ranking under class imbalance, F1 depends on a threshold, and AUROC can look strong when practical precision remains low. Review-budget recall is closer to an investigation queue but does not measure case resolution, deterrence, or financial loss. Equation (6) uses a hypothetical 5:1 cost ratio. The IBM labels describe synthetic laundering scenarios, whereas Elliptic and DGraphFin use different real-data entities and label processes. None is a complete proxy for a private bank's AML workflow.

External validity. Validated predictive evidence covers two real public graphs and four Small/Medium variants of a synthetic generator. It does not cover a private-bank deployment, IBM Large, or Medium GINE. The T4 memory boundary may move with sampling, implementation, or hardware. The tested

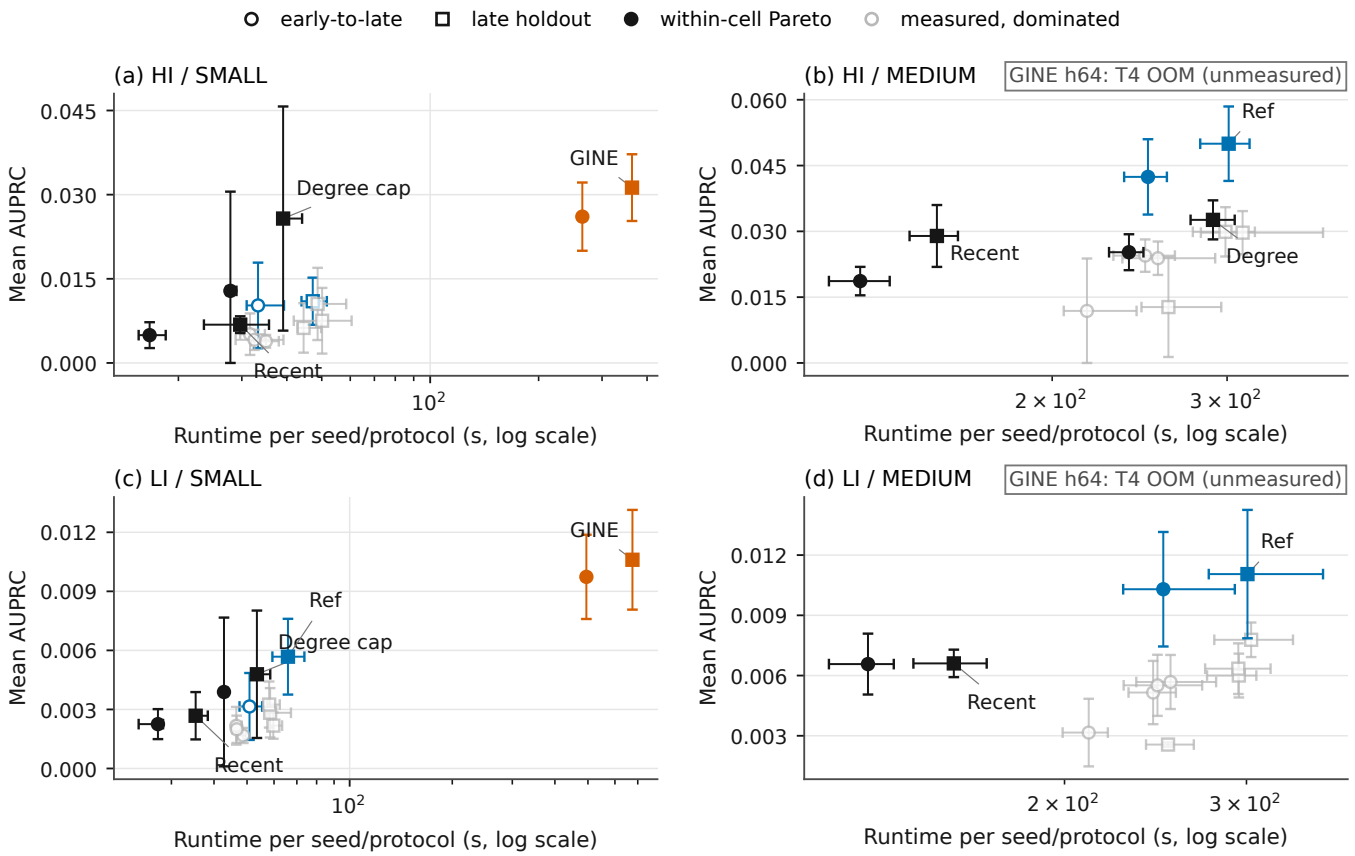

Fig. 6. Configuration-specific AUPRC and elapsed time under the recorded IBM envelope. Pareto status is computed only within a variant–protocol cell. Medium GINE and both Large variants remain nonnumeric and outside predictive ordering.

architectures and constructions are informative controls, not an exhaustive survey of temporal or heterogeneous GNNs. Dataset access and licensing also limit exact independent reruns.

Statistical conclusion validity. Ten seeds permit paired analysis but still leave uncertainty, particularly on DGraphFin and the low-intensity IBM variants. Very small absolute AUPRC changes can have large relative values at prevalence below 0.2%. Bootstrap intervals describe seed/cell variability in the observed grid, not sampling from all institutions or time periods. Deterministic library baselines can repeat exactly across nominal seeds; their ten rows document repeated pipeline evaluation, not ten stochastic fits. We use Holm correction for declared analysis families, keep descriptive rank correlations separate, and never pool unequal feasible sets. The four fixed IBM contexts and the six GraphSafe contexts are averaged within seed before inference; Small-only GINE means are not compared as though Medium were observed. Context-specific IBM rows remain visible because blocking can hide heterogeneity.

D. Ethics and Intended Use

Fraud flags can impose investigation costs, delay legitimate transactions, and expose people or organizations to adverse action. False negatives also carry social and financial costs. The reported review budgets are intended for human-in-the-loop triage, not automatic punitive decisions. The study does not evaluate demographic fairness because the validated artifacts do not contain an appropriate fairness surface; absence of such an analysis is not evidence of fairness. Synthetic IBM data reduces direct privacy exposure but cannot establish realism. Any operational use requires institution-specific validation, governance, appeal procedures, security review, and lawful data handling.

X. Conclusion

FraudShiftBench treats an empirical result as a claim under a deployment contract, not as a score attached only to a model and dataset. This framing changes the scientific conclusion in the validated evidence. On Elliptic, a graph-visibility intervention changes the leading AUPRC model from GraphSAGE to GCN; on DGraphFin, GraphSAGE remains first but loses 21.5% relative AUPRC. IBM AML-Data shows a second kind of instability: ranking and fixed-threshold metrics select different configurations in 11 of 16 feasible contexts, and construction tradeoffs change with scale. Resource guards leave Large and Medium GINE outside predictive ordering rather than assigning them implicit losses.

The support validator makes these boundaries executable. It rejected incomplete, prediction-missing, integrity-failed, construct-invalid, resource-widened, and contradicted claims in

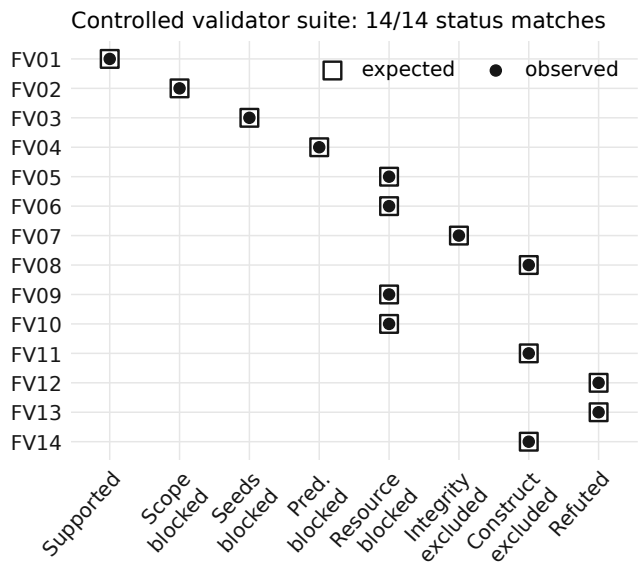

Fig. 7. Expected and observed outcomes for all 14 controlled mutations. The suite checks declared support conditions; it does not prove scientific truth or ontology completeness.

all 14 controlled cases. It also exposed three metadata labels that represented duplicate runs rather than distinct temporal stresses. The GraphSafe case illustrates the same discipline at the decision level: DGraphFin summaries are favorable, Elliptic favors simple averaging, and no comparison survives Holm correction across the declared 48-test family.

The central result is therefore conditional but actionable. Fraud benchmarks should report the contract that produced a ranking, separate ranking from operating-point utility, and keep unmeasured cells visible without turning them into performance evidence.

REFERENCES

[1] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLOS ONE*, 2015.

[2] A. C. Bahnsen, D. Aouada, and B. Ottersten, "Example-Dependent Cost-Sensitive Decision Trees," *Expert Systems with Applications*, 2015.

[3] M. A. Siddiqui, A. Fern, T. G. Dietterich, R. Wright, A. Theriault, and D. W. Archer, "Feedback-Guided Anomaly Discovery via Online Optimization," in *KDD*, 2018.

[4] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," KDD 2019 Workshop on Anomaly Detection in Finance; arXiv, 2019, arXiv:1908.02591.

[5] X. Huang, Y. Yang, Y. Wang, C. Wang, Z. Zhang, J. Xu, L. Chen, and M. Vazirgiannis, "DGraph: A Large-Scale Financial Dataset for Graph Anomaly Detection," in *NeurIPS Datasets and Benchmarks*, 2022.

[6] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic Synthetic Financial Transactions for Anti-Money Laundering Models," in *NeurIPS Datasets and Benchmarks*, 2023.

[7] G. C. Cawley and N. L. C. Talbot, "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *Journal of Machine Learning Research*, 2010. [Online]. Available: https://jmlr.org/beta/papers/v11/cawley10a.html

[8] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in Data Mining: Formulation, Detection, and Avoidance," *ACM Transactions on Knowledge Discovery from Data*, 2012.

[9] S. Kapoor and A. Narayanan, "Leakage and the Reproducibility Crisis in Machine-Learning-Based Science," *Patterns*, 2023.

[10] M. Dehghani, Y. Tay, A. A. Gritsenko, Z. Zhao, N. Houlsby, F. Diaz, D. Metzler, and O. Vinyals, "The Benchmark Lottery," arXiv/OpenReview, 2021, arXiv:2107.07002.

[11] F. Poursafaei, S. Huang, K. Pelrine, and R. Rabbany, "Towards Better Evaluation for Dynamic Link Prediction," in *NeurIPS Datasets and Benchmarks*, 2022.

[12] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy, and M. J. Kochenderfer, "BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices," in *NeurIPS Datasets and Benchmarks*, 2024.

[13] S. Huang, F. Poursafaei, J. Danovitch, M. Fey, W. Hu, E. Rossi, J. Leskovec, M. Bronstein, G. Rabusseau, and R. Rabbany, "Temporal Graph Benchmark for Machine Learning on Temporal Graphs," in *NeurIPS Datasets and Benchmarks*, 2023.

[14] J. Gastinger, S. Huang, M. Galkin, E. Loghmani, A. Parviz, F. Poursafaei, J. Danovitch, E. Rossi, I. Koutis, H. Stuckenschmidt, R. Rabbany, and G. Rabusseau, "TGB 2.0: A Benchmark for Learning on Temporal Knowledge Graphs and Heterogeneous Graphs," in *NeurIPS Datasets and Benchmarks*, 2024.

[15] Q. Huang, X. Wang, S. X. Rao, Z. Han, Z. Zhang, Y. He, Q. Xu, Y. Zhao, Z. Zheng, and J. Jiang, "BenchTemp: A General Benchmark for Evaluating Temporal Graph Neural Networks," in *ICDE*, 2024.

[16] J. Tang, F. Hua, Z. Gao, P. Zhao, and J. Li, "GADBench: Revisiting and Benchmarking Supervised Graph Anomaly Detection," in *NeurIPS Datasets and Benchmarks*, 2023.

[17] F. Hua, Y. Qi, Z. Wei, Y. Tian, C. Xu, X. Wu, J. Li, and J. Guo, "BAG: Benchmarking Anomaly Detection on Dynamic Graphs," in *AAAI*, 2026.

[18] Z. Liu, Y. Dou, P. S. Yu, Y. Deng, and H. Peng, "Alleviating the Inconsistency Problem of Applying Graph Neural Network to Fraud Detection," in *SIGIR*, 2020.

[19] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing Graph Neural Network-based Fraud Detectors against Camouflaged Fraudsters," in *CIKM*, 2020.

[20] Y. Liu, X. Ao, Z. Qin, J. Chi, J. Feng, H. Yang, and Q. He, "Pick and Choose: A GNN-based Imbalanced Learning Approach for Fraud Detection," in *The Web Conference*, 2021.

[21] J. Tang, J. Li, Z. Gao, and J. Li, "Rethinking Graph Neural Networks for Anomaly Detection," in *ICML*, 2022. [Online]. Available: https://proceedings.mlr.press/v162/tang22b.html

[22] B. Egressy, L. von Niederhäusern, J. Blanuša, E. Altman, R. Wattenhofer, and K. Atasu, "Provably Powerful Graph Neural Networks for Directed Multigraphs," in *AAAI*, 2024.

[23] R. Wu, B. Ma, H. Jin, W. Zhao, W. Wang, and T. Zhang, "GRANDE: a neural model over directed multigraphs with application to anti-money laundering," arXiv, 2023, arXiv:2302.02101.

[24] J. Lin, X. Guo, Y. Zhu, S. Mitchell, E. Altman, and J. Shun, "FraudGT: A Simple, Effective, and Efficient Graph Transformer for Financial Fraud Detection," in *ICAIF*, 2024.

[25] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *ICLR*, 2017, arXiv:1609.02907.

[26] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *NeurIPS*, 2017, arXiv:1706.02216.

[27] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *ICLR*, 2018, arXiv:1710.10903.

[28] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *ICLR*, 2019, arXiv:1810.00826.

[29] J. Blanuša, M. C. Baraja, A. Anghel, L. von Niederhäusern, E. Altman, H. Pozidis, and K. Atasu, "Graph Feature Preprocessor: Real-time Subgraph-based Feature Extraction for Financial Crime Detection," in *ICAIF*, 2024.

[30] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud Detection: A Systematic Literature Review of Graph-Based Anomaly Detection Approaches," *Decision Support Systems*, 2020.

[31] J. Zhou, S. Huang, Q. Qing, Z. Yuan, H. Huang, Z. Xu, M. Hou, X. Zhang, R. Luo, and I. Lee, "GAD in the Wild: Benchmarking Graph Anomaly Detection under Realistic Deployment Challenges," arXiv, 2026, arXiv:2605.07133.

[32] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open Graph Benchmark: Datasets for Machine Learning on Graphs," in *NeurIPS*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/fb60d411a5c5b72b2e7d3527cfc84fd0-Abstract.html

[33] S. Gui, X. Li, L. Wang, and S. Ji, "GOOD: A Graph Out-of-Distribution Benchmark," in *NeurIPS Datasets and Benchmarks*, 2022.

[34] K. Gorman and S. Bedrick, "We Need to Talk about Standard Splits," in *ACL*, 2019.
[35] Y. Oh, "State-of-the-Art Claims Require State-of-the-Art Evidence," arXiv, 2026, arXiv:2605.17273.
[36] E. M. Bender and B. Friedman, "Data Statements for Natural Language Processing," *Transactions of the Association for Computational Linguistics*, 2018.
[37] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. D. III, and K. Crawford, "Datasheets for Datasets," *Communications of the ACM*, 2021.
[38] M. Pushkarna, A. Zaldivar, and O. Kjartansson, "Data Cards: Purposeful and Transparent Dataset Documentation for Responsible AI," in *FAccT*, 2022.
[39] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model Cards for Model Reporting," in *FAT**, 2019.
[40] A. Sokol, E. Daly, M. Hind, D. Piorkowski, X. Zhang, N. Moniz, and N. V. Chawla, "BenchmarkCards: Standardized Documentation for Large Language Model Benchmarks," in *NeurIPS Datasets and Benchmarks*, 2025, arXiv:2410.12974.
[41] F. Bordes, C. Ross, J. T. Kao, E. Spiliopoulou, and A. Williams, "Eval Factsheets: A Structured Framework for Documenting AI Evaluations," arXiv, 2025, arXiv:2512.04062.
[42] A. Z. Jacobs and H. Wallach, "Measurement and Fairness," in *FAccT*, 2021.
[43] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *ICML*, 2017. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html
[44] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the Outputs of a Classifier to New a Priori Probabilities: A Simple Procedure," *Neural Computation*, 2002.
[45] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," in *NeurIPS*, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html
[46] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *The Annals of Statistics*, 2001.
[47] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for Pre-training Graph Neural Networks," in *ICLR*, 2020, arXiv:1905.12265.
[48] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, 1979.
[49] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, 1979. [Online]. Available: https://www.jstor.org/stable/4615733